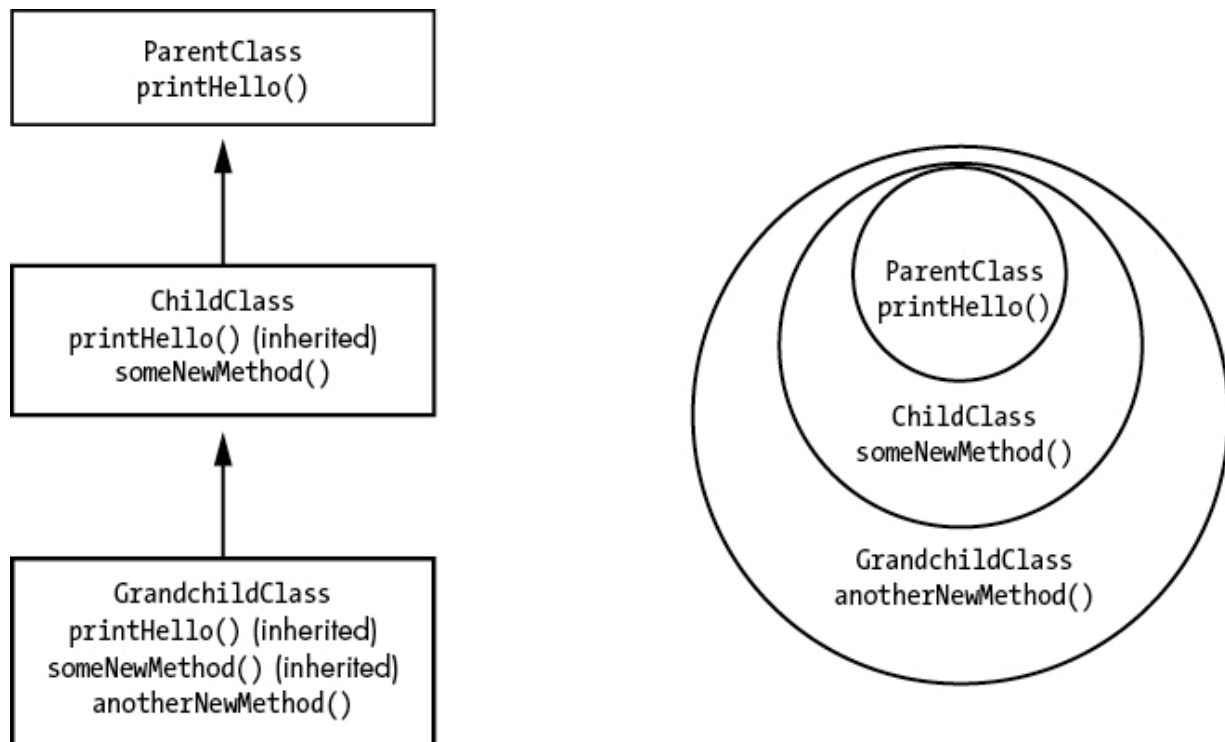




[Figure 16-1](#): A hierarchical diagram (left) and Venn diagram (right) showing the relationships between the three classes and the methods they have

It's common to say that parent-child classes represent “is a” relationships. A `ChildClass` object is a `ParentClass` object because it has all the same methods that a `ParentClass` object has, including some additional methods it defines. This relationship is one way: a `ParentClass` object is not a `ChildClass` object. If a `ParentClass` object tries to call `someNewMethod()`, which only exists for `ChildClass` objects (and the subclasses of `ChildClass`), Python raises an `AttributeError`.

Programmers often think of related classes as having to fit into some real-world “is a” hierarchy. OOP tutorials commonly have parent, child, and grandchild classes of `Vehicle`►`FourWheelVehicle`►`Car`, `Animal`►`Bird`►`Sparrow`, OR `Shape`►`Rectangle`►`Square`. But remember that the primary purpose of inheritance is code reuse. If your program needs a class with a set of methods that is a complete superset of some other class's methods, inheritance allows you to avoid copying and pasting code.

We also sometimes call a child class a *subclass* or *derived class* and call a parent class the *super class* or *base class*.

Overriding Methods